

Plan: Incorporate docs_chain as a Lava Submodule

Date: 2026-06-02 **Author:** Stream C subagent (prep/analysis — read-only on parent) **Strategy chosen by operator:** full-cascade-up-front (bring docs_chain upstream to parity BEFORE git submodule add) **Target path in parent:** submodules/docs_chain/ **Upstream:** git@github.com:vasic-digital/docs_chain.git **Pinned HEAD:** 02eb81be **Classification:** project-specific (docs_chain adoption is Lava-specific; the cascade content authored into it is universal per §6.AD)

A. Probe Findings — docs_chain Actual Structure

Clone executed successfully:

```
git clone --depth 1 git@github.com:vasic-digital/docs_chain.git /tmp/docs_chain_probe_$$
```

Root-level inventory

Path	Present?
README.md	YES
go.mod	YES (module digital.vasic.docs_chain, go 1.25.0)
go.sum	YES
cmd/docs_chain/	YES (CLI binary)
internal/	YES (hash, graph, adapter, orchestrator, config, state, runner)
scripts/e2e.sh	YES
docs/	YES (ARCHITECTURE.md, USER_GUIDE.md, CONFIG_SCHEMA.md, USE_CASE_CATALOGUE.md, CONSTITUTION_INTEGRATION.md + .docx/.html/.pdf siblings)
qa-results/	YES

Path	Present?
README.html, README.pdf, README.docx	YES
CLAUDE.md	MISSING
AGENTS.md	MISSING
CONSTITUTION.md	MISSING
QWEN.md	MISSING
helix-deps.yaml	MISSING
install_upstreams.sh	MISSING
Upstreams/ or upstre ams/	MISSING

Repo purpose (from README.md)

Docs Chain is a universal, Go-implemented, **bidirectional document-and-database dependency-propagation engine**. It is a vasic-digital submodule and the intended successor to ad-hoc documentation-sync scripts across the vasic-digital / HelixConstitution ecosystem. Phases 1-5 (core DAG, content-hash engine, node adapters, propagation orchestrator, config-driven CLI, comprehensive test suite) are IMPLEMENTED and tested. Phases 6-7 (constitution-submodule distribution, ATMOSphere wiring) are OPERATOR-GATED / PLANNED.

Go module: digital.vasic.docs_chain. Third-party deps: fsnotify, golang.org/x/net, gopkg.in/yaml.v3, modernc.org/sqlite. No own-org submodule deps.

Remote configuration

docs_chain currently has ONLY the origin remote pointing to GitHub:

```
origin  git@github.com:vasic-digital/docs_chain.git (fetch)
origin  git@github.com:vasic-digital/docs_chain.git (push)
```

GitLab mirror does NOT yet exist. This is a \$6.W concern — addressed in section D.

B. Parity Gaps vs Parent Gate Bar

The parent's pre-push gate (scripts/check-canonical-root-and-upstreams.sh + scripts/check-constitution.sh) enforces the following requirements for every submodule under submodules/*. All five are currently MISSING from docs_chain:

Gate	Requirement	docs_chain current	Status
§11.4.35 (CM-CANONICAL-ROOT-CLARITY)	CLAUDE.md opens with ## INHERITED FROM constitution/ CLAUDE.md within first 40 lines	No CLAUDE.md exists	MISSING
§11.4.35 (CM-CANONICAL-ROOT-CLARITY)	AGENTS.md opens with ## INHERITED FROM constitution/ AGENTS.md within first 40 lines	No AGENTS.md exists	MISSING
§11.4.35 (CM-CANONICAL-ROOT-CLARITY)	CONSTITUTION.md opens with ## INHERITED FROM constitution/ Constitution.md within first 40 lines	No CONSTITUTION.md exists	MISSING
§6.AD pointer-block (check-constitution.sh)	CLAUDE.md, AGENTS.md, CONSTITUTION.md carry ## INHERITED FROM constitution/ block	None exist	MISSING
§11.4.36 (CM-INSTALL-UPSTREAMS-RAN)	install_upstreams.sh (or scripts/ install_upstreams.sh) present at submodule root	Not present	MISSING
CM-HELIX-DEPS-MANIFEST	helix-deps.yaml present at submodule root	Not present	MISSING
QWEN.md	Required by §6.AD cascade + QWEN.md inheritance mandate	Not present	MISSING
§6.W (two-mirror rule)	GitLab remote must exist alongside GitHub	Only GitHub	MISSING

Additionally: Upstreams/ directory with GitHub.sh + GitLab.sh recipe files is required for install_upstreams.sh to work and for \$6.W push fan-out.

C. Exact File Contents to Author INTO docs_chain Upstream

All seven files below MUST be authored into the docs_chain repo (pushed to both GitHub and GitLab — after the GitLab mirror is created) before git submodule add is run in the parent.

C.1 — helix-deps.yaml

```
# helix-deps.yaml – docs_chain Submodule-Dependency Manifest
# Per HelixConstitution §11.4.31 (Submodule-Dependency-Manifest
#   Mandate / CONST-054).
#
# docs_chain is a leaf Go submodule (digital.vasic.docs_chain)
#   providing a
#   bidirectional document-and-database dependency-propagation
#   engine. It has
#   ZERO own-org submodule dependencies – pure Go module + standard
#   3rd-party
#   deps (fsnotify, yaml.v3, modernc.org/sqlite) declared in go.mod.
#
# Schema reference: constitution/Constitution.md §11.4.31.
# Last updated: 2026-06-02 (Lava incorporation cascade).

schema_version: 1

# docs_chain has no own-org dependencies. The empty deps array is
#   the
#   honest manifest for a leaf submodule. Future agents reading this
#   file have positive evidence docs_chain is incorporable as a peer
#   with
#   zero transitive own-org-dep recursion.
deps: []

transitive_handling:
  recursive: true
  conflict_resolution: operator-required

# docs_chain is a Go module – language_specific_subtree is false
#   because
#   the entire submodule is the Go subtree (no inner non-Go subtrees
#   requiring §11.4.29 snake_case exemption).
language_specific_subtree: false
```

C.2 — CLAUDE.md

CLAUDE.md – docs_chain Module

INHERITED FROM constitution/CLAUDE.md

All rules in ``constitution/CLAUDE.md`` (and the ``constitution/Constitution.md`` it references) apply unconditionally. This file's rules below extend them – they MUST NOT weaken any inherited rule. See parent project's root ``CLAUDE.md`` §6.AD for the Lava-specific incorporation context (29th §6.L cycle, 2026-05-14) and §6.AD-debt for the implementation-gap inventory. Use ``constitution/find_constitution.sh`` from the parent project root to resolve the absolute path of the constitution submodule from any nested location.

Definition of Done

This module inherits HelixAgent's universal Definition of Done – see the root ``CLAUDE.md`` and ``docs/development/definition-of-done.md``. In one line: ****no** task is done without pasted output from a real run of the real system in the same session as the change.****** Coverage and green suites are not evidence.

Acceptance demo for this module

```
```bash
cd docs_chain
GOMAXPROCS=2 nice -n 19 go test -race -count=1 ./...
```

Expect: PASS; exercises all phases 1–5 of the DAG, hash, adapter, orchestrator, config, state, runner, and CLI packages.

## Overview

digital.vasic.docs\_chain is a universal, Go-implemented, bidirectional document-and-database dependency-propagation engine. It detects changes by content hash and propagates them through a DAG of registered chain members atomically.

**Module:** digital.vasic.docs\_chain (Go 1.25+)

## Build & Test

```
go build ./...
go test ./... -race -count=1
go build -o ./docs_chain ./cmd/docs_chain
```

## Commit Style

Conventional Commits: feat(graph): add early-cutoff optimisation

## No sudo/su (§6.U)

ALL operations MUST run at local user level ONLY. No sudo or su in any committed script, Makefile, or tool call.

## Host Power Management — Hard Ban

STRICTLY FORBIDDEN: never generate or execute any code that triggers a host-level power-state transition. See parent CLAUDE.md §Host Machine Stability Directive for the full forbidden command list.

## §6.S — Continuation Document Maintenance (inherited)

See parent root CLAUDE.md §6.S. The parent docs/CONTINUATION.md is the single-file source-of-truth handoff. Every commit that changes this submodule's phase status or ships a release artifact MUST update docs/CONTINUATION.md in the SAME parent commit.

## §6.W — GitHub + GitLab Only Remotes (inherited)

See parent root CLAUDE.md §6.W. Only GitHub (vasic-digital/docs\_chain) and GitLab (vasic-digital/docs\_chain) are permitted as Git remotes. All push fan-out MUST go through both.

# Anti-Bluff Testing Pact (inherited §6.J / §6.L / Sixth + Seventh Laws)

Every test, every CI gate, has exactly one job: confirm the feature works for a real user end-to-end. CI green is necessary, NEVER sufficient. Tests must guarantee the product works — anything else is theatre. See parent root CLAUDE.md §6.J + §6.L for the full mandate.

### C.3 – `AGENTS.md`

```markdown

AGENTS.md – docs_chain Module Multi-Agent Coordination

INHERITED FROM constitution/AGENTS.md

All rules in `constitution/AGENTS.md` (and the `constitution/Constitution.md` it references) apply unconditionally. This file's rules below extend them – they MUST NOT weaken any inherited rule. See parent project's root `CLAUDE.md` §6.AD for the Lava-specific incorporation context (29th §6.L cycle, 2026-05-14) and §6.AD-debt for the implementation-gap inventory.

Module Identity

- **Module:** `digital.vasic.docs\_chain`
- **Role:** Bidirectional document-and-database dependency-propagation engine
- **Packages:** `internal/hash`, `internal/graph`, `internal/adapter`,
`internal/orchestrator`, `internal/config`, `internal/state`,
`internal/runner`, `cmd/docs\_chain`
- **Go Version:** 1.25+

Agent Responsibilities

The docs_chain agent owns all packages in this module. It is responsible for:

1. **Core engine** (`internal/hash`, `internal/graph`) – content-hash change detection, DAG topology, Kahn ordering, early-cutoff recomputation, sync conflict resolution.
2. **Node adapters** (`internal/adapter`) – FileAdapter (markdown, html, pdf, docx), SQLiteAdapter (canonical row dump, byte-stable),

FileStore.

3. ****Propagation orchestrator**** (`internal/orchestrator`) – atomic multi-file propagation with rollback on error, cycle-guard, sync-conflict surfacing.
4. ****Config + state + runner**** (`internal/config`, `internal/state`, `internal/runner`) – per-context YAML loading, state.json baseline, full orchestration pipeline.
5. ****CLI**** (`cmd/docs\_chain`) – `sync`, `verify`, `doctor`, `graph`, `watch` subcommands with exit-code contract.

Cross-Agent Coordination

This submodule is standalone. When consumed by a parent project (e.g. Lava), the agent works at the parent level to register the consuming project's `.docs\_chain/contexts/<name>.yaml` files; changes to the engine itself go upstream to this repo first, then a parent pin bump follows.

Anti-Bluff Mandate

Every test added to this module MUST satisfy all Sixth + Seventh Law clauses inherited from the parent. See parent root `CLAUDE.md` §6.J / §6.L.

C.4 — CONSTITUTION.md

docs_chain – Constitution

INHERITED FROM constitution/Constitution.md

All rules in `constitution/Constitution.md` (and the `constitution/Constitution.md` it references) apply unconditionally. This file's rules below extend them – they MUST NOT weaken any inherited rule. See parent project's root `CLAUDE.md` §6.AD for the Lava-specific incorporation context (29th §6.L cycle, 2026-05-14) and §6.AD-debt for the implementation-gap inventory.

> ****Status:**** Active. This document is the authoritative rule set for the
 > `docs\_chain` module. When a rule here conflicts with
 > `CLAUDE.md`, `AGENTS.md`,
 > or any guide, the Constitution wins.

Module-Level Rules

All constitutional rules from the parent and the constitution submodule apply unconditionally. Module-specific extensions:

1. ****No faking transform success.**** When pandoc / weasyprint / any required tool is absent, the run MUST return a ``ToolAbsentError`` and roll back – never write a partial or empty output file. (Composes with Sixth Law clause 3.)
2. ****Sync conflicts are surfaced, never silently resolved.**** A ``both-dirty`` sync pair MUST produce a ``ConflictError`` exit code (2); the caller decides resolution. Never auto-pick a winner. (Composes with §9.2 atomicity.)
3. ****Content-hash change detection only, never mtime.**** Any reversion to mtime-based change detection is a constitutional violation. The hash is the authority.
4. ****Anti-Bluff Forensic Anchor**** (cascaded from parent CONSTITUTION.md §Article XI §11.9): the bar for shipping is not "tests pass" but "users can use the feature." Every PASS MUST carry positive runtime evidence captured during execution.

Amendment Process

Constitution amendments require:

1. Written proposal with rationale
2. Challenge demonstrating the need
3. Approval by project architect
4. Update to this file and cascade to parent governance docs

C.5 — QWEN.md

QWEN.md – Qwen Code context for the docs_chain module

This file is read by Qwen Code as its module-context file. It is the Qwen Code counterpart of CLAUDE.md and AGENTS.md for this module, and it is a pointer: there is one canonical agent-instruction file per scope.

Read CLAUDE.md – it is mandatory

This module's canonical agent-instruction file is CLAUDE.md in this directory. Before doing any work in this module, open and read CLAUDE.md. All rules in CLAUDE.md apply unconditionally to Qwen Code sessions in this module.

Inheritance pointer

INHERITED FROM constitution/CLAUDE.md

(Qwen Code: the above heading is the §11.4.35 inheritance marker – the parent ``CLAUDE.md`` inherits from ``constitution/CLAUDE.md``, and this module's CLAUDE.md inherits by transitivity. Read the parent project's ``constitution/CLAUDE.md`` and ``constitution/Constitution.md`` for the full universal rule set.)

C.6 — install_upstreams.sh

```
#!/bin/bash
#
# install_upstreams.sh – Configure git remotes for all upstream
#                        repositories.
#
# Reads UPSTREAMABLE_REPOSITORY from each .sh file in the
# Upstreams/
# directory and adds them as git remotes. Existing remotes are
# updated.
#
# Usage: ./install_upstreams.sh [--push] [--dry-run]
#
# Options:
#   --push      Push current branch to all upstreams after
#               configuration
#   --dry-run   Show what would be done without making changes
#
# Inheritance: §11.4.36 (CONST-056), §6.W (GitHub + GitLab only).
# Classification: project-specific (vasic-digital +
#               HelixDevelopment infra).
```

set -e

SCRIPT_DIR="\$(cd "\$(dirname "\${BASH_SOURCE[0]}")" && pwd)"

```

UPSTREAMS_DIR="${SCRIPT_DIR}/Upstreams"

if [[ ! -d "${UPSTREAMS_DIR}" ]]; then
    echo "Error: Upstreams directory not found at ${UPSTREAMS_DIR}" >&2
    echo "Create Upstreams/ with GitHub.sh and GitLab.sh recipe files." >&2
    exit 1
fi

PUSH=false
DRY_RUN=false

while [[ $# -gt 0 ]]; do
    case $1 in
        --push)    PUSH=true;    shift ;;
        --dry-run) DRY_RUN=true; shift ;;
        *) echo "Unknown option: $1" >&2; echo "Usage: $0 [--push] [--dry-run]" >&2; exit 1 ;;
    esac
done

echo "=== Configuring upstream git remotes ==="
echo "Upstreams directory: ${UPSTREAMS_DIR}"
echo

for script in "${UPSTREAMS_DIR}/*.sh; do
    [[ -f "$script" ]] || continue
    upstream_name="$(basename "$script" .sh)"
    repo_url="$(bash -c "source \"$script\" && echo \"\$UPSTREAMABLE_REPOSITORY\"" )"
    if [[ -z "$repo_url" ]]; then
        echo "Warning: $script does not export UPSTREAMABLE_REPOSITORY" >&2
        continue
    fi
    echo "Processing $upstream_name..."
    echo "  Repository URL: $repo_url"
    if [[ "$DRY_RUN" == "true" ]]; then
        echo "  [DRY RUN] Would configure remote: $upstream_name -> $repo_url"
        continue
    fi
    if git remote add "$upstream_name" "$repo_url" 2>/dev/null; then
        echo "  Added remote: $upstream_name"
    else
        git remote set-url "$upstream_name" "$repo_url"
        echo "  Updated remote: $upstream_name"
    fi
done

echo

```

```

echo "=== Current git remotes ==="
git remote -v

if [[ "$PUSH" == "true" ]]; then
    echo
    echo "=== Pushing to all upstreams ==="
    current_branch="$(git branch --show-current)"
    [[ -z "$current_branch" ]] && { echo "Error: Not on a branch"
    >&2; exit 1; }
    for remote in $(git remote); do
        echo "Pushing to $remote/$current_branch..."
        if [[ "$DRY_RUN" == "true" ]]; then
            echo " [DRY RUN] Would run: git push $remote
            $current_branch"
        else
            git push "$remote" "$current_branch" || echo "
            Warning: push to $remote failed"
        fi
    done
fi

echo
echo "=== Done ==="

```

C.7 — Upstreams/GitHub.sh

```

#!/bin/bash

export UPSTREAMABLE_REPOSITORY="git@github.com:vasic-digital/
docs_chain.git"

```

C.8 — Upstreams/GitLab.sh

```

#!/bin/bash

export UPSTREAMABLE_REPOSITORY="git@gitlab.com:vasic-digital/
docs_chain.git"

```

D. §6.W Concern — Missing GitLab Mirror

docs_chain currently has **only** the GitHub remote. Per §6.W, every vasic-digital-owned submodule MUST mirror to both GitHub and GitLab (CLI parity). Before the cascade commit can be pushed to "all upstreams", the GitLab mirror must be created.

Required pre-cascade step (OPERATOR action):

```
# Create the GitLab mirror (requires glab CLI authenticated to
    vasic-digital org)
glab repo create vasic-digital/docs_chain --private --remote-name
    gitlab \
    --description "Bidirectional document-and-database dependency-
        propagation engine"
# Then from the docs_chain working tree:
git remote add gitlab git@gitlab.com:vasic-digital/docs_chain.git
git push gitlab main
```

After this, docs_chain has both origin (GitHub) and gitlab remotes. The install_upstreams.sh script in C.6 + the Upstreams/GitHub.sh + Upstreams/GitLab.sh recipes in C.7–C.8 take over from that point.

E. Step-by-Step Landing Sequence for the Orchestrator

The ORCHESTRATOR runs these steps in order. Do NOT git submodule add in the parent until step 4 is complete.

Step 1: Create GitLab mirror (OPERATOR action required)

```
# From any working directory with glab authenticated:
glab repo create vasic-digital/docs_chain --private \
    --description "Bidirectional doc-and-database dependency-
        propagation engine"
# Confirm URL: git@gitlab.com:vasic-digital/docs_chain.git
```

Step 2: Clone docs_chain and author the cascade

```
git clone git@github.com:vasic-digital/docs_chain.git /tmp/
    docs_chain_cascade
cd /tmp/docs_chain_cascade

# 2a. Author all new files (content from section C above):
# Create helix-deps.yaml (C.1)
# Create CLAUDE.md (C.2)
# Create AGENTS.md (C.3)
# Create CONSTITUTION.md (C.4)
# Create QWEN.md (C.5)
# Create install_upstreams.sh (C.6) + chmod +x
# Create Upstreams/GitHub.sh (C.7) + chmod +x
# Create Upstreams/GitLab.sh (C.8) + chmod +x

# 2b. Wire the GitLab remote into this working clone:
git remote add gitlab git@gitlab.com:vasic-digital/docs_chain.git
```

```
# 2c. Run install_upstreams to confirm wiring:
./install_upstreams.sh --dry-run
```

Step 3: Commit and push cascade to docs_chain (both upstreams)

```
cd /tmp/docs_chain_cascade
```

```
git add helix-deps.yaml CLAUDE.md AGENTS.md CONSTITUTION.md
      QWEN.md \
      install_upstreams.sh Upstreams/GitHub.sh Upstreams/
      GitLab.sh
```

```
git commit -m "$(cat <<'EOF'
feat(cascade): add HelixConstitution governance + helix-deps.yaml
for Lava incorporation
```

```

  Authored in preparation for addition as a Lava submodule
    (submodules/docs_chain/).
  Per §6.AD full-cascade-up-front strategy: the parent's pre-push
    gate requires
  helix-deps.yaml (CM-HELIX-DEPS-MANIFEST), §6.AD inheritance
    pointer-blocks in
  CLAUDE.md / AGENTS.md / CONSTITUTION.md / QWEN.md, and
    install_upstreams.sh
  (CM-INSTALL-UPSTREAMS-RAN) to be present BEFORE git submodule add
    is executed.
```

Files added:

- helix-deps.yaml: leaf submodule manifest (no own-org deps) per §11.4.31
- CLAUDE.md: §11.4.35 inheritance pointer + module-local instructions
- AGENTS.md: §11.4.35 inheritance pointer + multi-agent coordination
- CONSTITUTION.md: §11.4.35 inheritance pointer + module constitution
- QWEN.md: Qwen Code context pointer
- install_upstreams.sh: §11.4.36 remote-configuration script
- Upstreams/GitHub.sh: GitHub recipe (git@github.com:vasic-digital/docs_chain.git)
- Upstreams/GitLab.sh: GitLab recipe (git@gitlab.com:vasic-digital/docs_chain.git)

Classification: project-specific (Lava adoption); cascade content universal per §6.AD.

```
EOF
)"
```

```
# Push to BOTH remotes per §6.W:
```

```
git push origin main
git push gitlab main
```

```
# Verify $6.C convergence – both should report the same tip SHA:
echo "GitHub HEAD: $(git ls-remote git@github.com:vasic-digital/
docs_chain.git HEAD | cut -f1)"
echo "GitLab HEAD: $(git ls-remote git@gitlab.com:vasic-digital/
docs_chain.git HEAD | cut -f1)"
# Both MUST match.
```

```
# Record the new pinned SHA (the cascade commit's SHA):
CASCADE_SHA=$(git rev-parse HEAD)
echo "Pin this SHA in the parent: $CASCADE_SHA"
```

Step 4: Add the submodule in the parent (Lava) repo

```
cd /Users/milosvasic/Projects/Lava

# Fetch the pre-push gate passes:
git submodule add git@github.com:vasic-digital/docs_chain.git
submodules/docs_chain
cd submodules/docs_chain && git checkout "$CASCADE_SHA" &&
cd ../../

# Verify the parent pre-push gates pass for the new submodule:
./scripts/check-canonical-root-and-upstreams.sh
./scripts/check-constitution.sh
# Both MUST exit 0 with no violations.
```

Step 5: Commit the parent changes

```
cd /Users/milosvasic/Projects/Lava

# Stage only the submodule registration files:
git add .gitmodules submodules/docs_chain

# Update CONTINUATION.md ($6.S – same commit):
# Add docs_chain to $3 pin index: submodules/docs_chain @
$CASCADE_SHA
# Update $0 "Last updated" to today.
git add docs/CONTINUATION.md

git commit -m "$(cat <<'EOF'
feat(submodules): add docs_chain as pinned vasic-digital submodule

Adds git@github.com:vasic-digital/docs_chain.git pinned at
<CASCADE_SHA>
under submodules/docs_chain/. The submodule was brought to parity
before
this add: helix-deps.yaml (leaf, zero own-org deps) + $6.AD
pointer-blocks
in CLAUDE.md/AGENTS.md/CONSTITUTION.md/QWEN.md +
install_upstreams.sh +
```

Upstreams/GitHub.sh + Upstreams/GitLab.sh authored upstream in the cascade commit. Both the CM-HELIX-DEPS-MANIFEST and CM-INSTALL-UPSTREAMS-RAN gates pass; check-canonical-root-and-upstreams.sh exits 0.

docs_chain is a bidirectional document-and-database dependency-propagation engine (Go, modernc.org/sqlite, content-hash DAG, atomic propagation). Phases 1–5 implemented and tested; Phase 6 (constitution integration) and Phase 7 (ATMOSphere wiring) operator-gated/planned.

docs/CONTINUATION.md §3 updated with the new pin.

Classification: project-specific (this adoption is Lava-specific).
EOF
)"

```
# Push to both mirrors per §6.W:
./scripts/commit_all.sh # or: git push origin master && git push
                           gitlab master
```

Step 6: Post-add verification

```
cd /Users/milosvasic/Projects/Lava
```

```
# Confirm the submodule is registered:
git submodule status submodules/docs_chain
# Must show: <CASCADE_SHA> submodules/docs_chain (...)
```

```
# Run the full constitution sweep:
./scripts/verify-all-constitution-rules.sh
# Must exit 0; §11.4.36 presence count increments by 1.
```

```
# Confirm §6.C convergence at parent level:
echo "GitHub parent HEAD: $(git ls-remote git@github.com:vasic-
    digital/Lava.git master | cut -f1)"
echo "GitLab parent HEAD: $(git ls-remote git@gitlab.com:vasic-
    digital/Lava.git master | cut -f1)"
# Both MUST match.
```

F. Summary of Parity Gaps (for at-a-glance tracking)

All 8 items below are OWED before `git submodule add` can run:

1. CLAUDE.md with `## INHERITED FROM constitution/CLAUDE.md` — **create** (content in C.2)

2. AGENTS.md with ## INHERITED FROM constitution/AGENTS.md — **create** (content in C.3)
 3. CONSTITUTION.md with ## INHERITED FROM constitution/Constitution.md — **create** (content in C.4)
 4. QWEN.md pointer file — **create** (content in C.5)
 5. helix-deps.yaml (leaf, deps: []) — **create** (content in C.1)
 6. install_upstreams.sh (executable) — **create** (content in C.6)
 7. Upstreams/GitHub.sh + Upstreams/GitLab.sh — **create** (content in C.7 / C.8)
 8. GitLab mirror git@gitlab.com:vasic-digital/docs_chain.git — **create via glab repo create** (OPERATOR action, §6.W)
-

STATUS: DONE